MGL 7230: Tests Logiciels

Chapitre #4

Tests Unitaires

Plan

- Contexte des tests unitaires
- Développement piloté par les tests (TDD)
- Conception de cas de test unitaire
- Automatisation des tests unitaires
- Qu'est-ce que Junit 5
- Fonctionnalités JUnit
- Architecture Junit 5
- Annotations et assertions JUnit 5
- Scalfolding
- Mockito

Besoin de tests unitaires

- Assure que les **unités** de programme **individuelles** fonctionnent comme prévu.

- **Unité** : peut être une classe ou une collection de classes dans un langage orienté objet (OO) tel que Java ou C#, ou une fonction ou une collection de fonctions dans un langage procédural tel que C.

- **Besoin de tests unitaires**
  - **Ajout de fonctionnalités :** toute nouvelle fonctionnalité introduite ne doit pas perturber les fonctionnalités existantes
  - **Tests de régression :** toute amélioration apportée doit garantir que les fonctionnalités précédemment fonctionnelles restent inchangées.
  - **Refactoring** : toute modification implémentée doit préserver la fonctionnalité existante sans introduire de problèmes.

Présentation des tests unitaires

- En général, **le programmeur** qui a conçu et codé l'unité est responsable de la création de ses cas de test.
  - En programmation en binôme, ces tâches peuvent être effectuées par deux programmeurs.

- Création de cas de test : diverses techniques peuvent être utilisées, telles que **le partitionnement par équivalence** ou les **modèles d'états finis du comportement**.

- **Adéquation des tests** : l'efficacité des tests unitaires peut être évaluée à l'aide de techniques d'adéquation des tests (par exemple, **couverture**, **tests de mutation**, etc.).

Contexte des tests unitaires

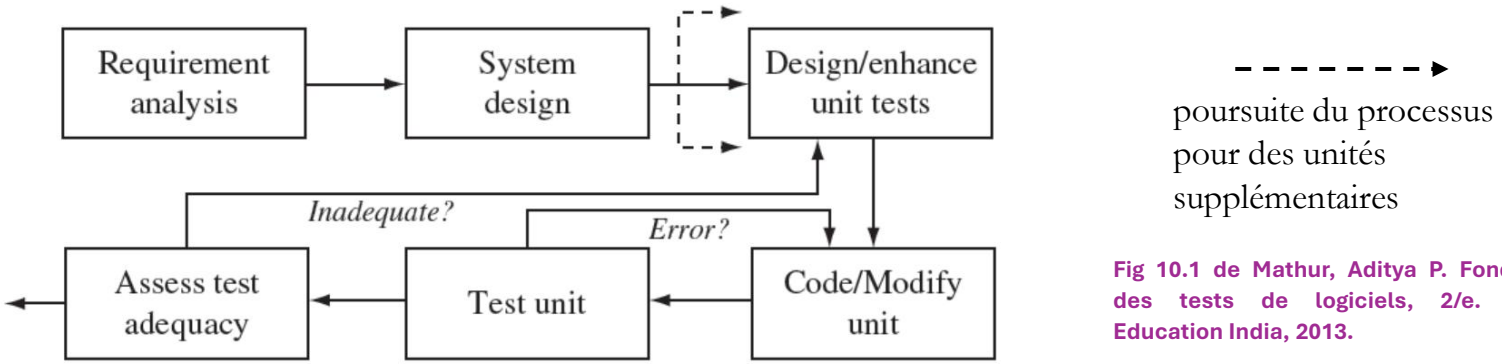


Fig 10.1 de Mathur, Aditya P. Fondements des tests de logiciels, 2/e. Pearson Education India, 2013.

- Le programmeur peut commencer à coder l'unité, puis concevoir les tests, ou peut d'abord concevoir les tests (comme indiqué sur la figure), puis coder l'unité correspondante (méthodologie de développement pilotée par les tests: TDD).
- Le développement incrémental consiste à tester l'unité en boucle : tester la partie développée, l'améliorer et la tester à nouveau jusqu'à ce que l'exactitude soit atteinte.
- La conception, le codage et les tests des tests sont répétés de manière itérative dans ce processus.

Développement piloté par les tests (TDD: Test-Drive Development)

- Compte tenu des **exigences**, il est possible de concevoir les tests avant de coder une unité.
- Il est en fait préférable d'écrire les tests **avant d'écrire le code que vous souhaitez tester**
- Lorsque les tests sont écrits en premier, vous avez **une idée plus claire** de ce qu'il faut faire lorsque vous écrivez les méthodes
- Étant donné que les tests sont écrits en premier, les méthodes sont nécessairement écrites pour être **testables**
- Écrire des tests en premier vous encourage à écrire des **méthodes plus simples et à usage unique**

Les trois lois du TDD

1. Vous ne pouvez pas écrire de code de production tant que le test défaillant n'existe pas.
2. Vous ne pouvez pas écrire plus de code de production que nécessaire pour échouer. Ne pas compiler équivaut à un échec.
3. Vous ne pouvez pas écrire plus de code de production que nécessaire pour réussir le test défaillant.

En bref : **test d'abord, échec d'abord, puis juste assez de code pour passer, en avançant par petits incréments.**

Cycle: **Red → Green → Refactor (implicite)**

Conception de cas de test

- Un cas de test est un ensemble ou une séquence d'entrées dans une unité testée, ainsi que la réponse attendue.
- Lorsque des cas de test sont générés avant le codage de l'unité, il faut utiliser une **méthode de génération de test en boîte noire**, par exemple le partitionnement d'équivalence.
- Un cas de test peut être codé comme un programme qui encapsule le cas de test, les entrées et les sorties, appelle l'unité avec les données d'entrée et vérifie si la réponse est conforme aux attentes.
  - Les cas de test codés permettent une réexécution facile du programme par rapport à un test (par exemple, lorsqu'une modification est apportée à l'unité).
- Des outils tels que JUnit permettent de coder plusieurs cas de test et d'exécuter automatiquement l'unité par rapport à ceux-ci.

Mettre en place des tests

Cas de test (test case) :
Les valeurs du cas de test, les valeurs de préfixe, les valeurs de suffixe et les résultats attendus nécessaires à une exécution et une évaluation complètes du logiciel testé

Ensemble de tests (test suite) :
Un ensemble de cas de test

Script de test exécutable :
Un cas de test préparé sous une forme qui doit être exécutée automatiquement sur le logiciel de test et produire un rapport

Conception de cas de test : exemple : type de triangles

- Le programme de classification des triangles prend trois entrées, a, b et c, représentant les longueurs des trois côtés d'un triangle.
- Il doit d'abord vérifier si les côtés forment ou non un triangle.
  - Si ce n'est pas le cas, une exception est levée, sinon le type de triangle correct est renvoyé.
- Supposons que les types de triangles suivants soient pris en compte : équilatéral (1), isocèle (2) et scalène (3).
- **Quatre cas de test sont nécessaires.**

Conception de cas de test : exemple : classificateur des triangles

- **Test 1 : une exception est levée lorsque l'entrée n'est pas valide.**

entrée : <a=−1, b=1, c=1>
Réponse attendue : Exception "Invalid input" must be raised.

- **Test 2:** Pour tester un triangle équilatéral.

entrée : <a=1, b=1, c=1>
Réponse attendue : 1

- **Test 3:** Pour tester un triangle isocèle.

entrée : <a=1, b=1, c=2>
Réponse attendue : 2

- **Test 4:** Pour tester un triangle quelconque.

entrée : <a=4, b=5, c=6>
Réponse attendue : 3

- **Dans TDD, ces tests seraient codés à l'aide d'un outil, par exemple JUnit, et exécutés sur un classificateur qui renvoie simplement, disons, 0, quelle que soit l'entrée.**
- **Le classificateur échouera à tous les cas de test. On peut maintenant développer progressivement le classificateur et tester chaque incrément.**
- **Progressivement, de plus en plus de tests réussiront jusqu'à ce que tous les tests soient exécutés**

Automatisation des tests unitaires

- Pour un système de grande taille, il peut y avoir des **milliers de tests unitaires**, qui peuvent être fastidieux à maintenir et à exécuter.

- Les cas de test unitaires **doivent être automatisés**.
  - Réduit le coût des tests de régression
  - Réduit les erreurs humaines
  - Réduit les écarts de qualité des tests entre les différents individus

- L'automatisation est obtenue automatiquement en :
  - Configurant un contexte de test,
  - Appelant chaque cas de test,
  - Vérifiant le résultat attendu correspondant et
  - Rapportant l'état de tous les tests.

Automatisation des tests unitaires

- Une fois que le code qui implémente une nouvelle fonctionnalité passe ses tests unitaires, ces tests sont alors appelés tests de « régression ».
  - Ces tests sont ensuite ajoutés à la base de code (avec le code) et sont réexécutés chaque fois qu'un nouveau code est ajouté à la base de code.
- Si un nouveau code fait échouer un ancien test, il est facile de savoir immédiatement que le nouveau code a enfreint une fonctionnalité déjà implémentée.
  - Il est primordial d'isoler le défaut le plus facilement possible !

Automatisation des tests unitaires

- Peut être combiné avec l'utilisation d'un référentiel de versions de logiciels (par exemple, Git, CVS, IBM Rational ClearCase)

- **Avant toute validation, des tests unitaires sont réappliqués** pour s'assurer que le code validé fonctionne toujours correctement.

Cadres d'automatisation des tests

Un ensemble d'hypothèses, de concepts et d'outils qui prennent en charge l'automatisation des tests

Exemples de frameworks de tests unitaires :

- JUnit for Java
- NUnit for all .Net languages such as C#

Qu'est-ce que JUnit ?

- Framework de test Java open source (junit.org) utilisé pour écrire et exécuter des tests automatisés répétitifs
- Outil standard pour **le développement piloté par les tests** en Java
- Il s'agit d'une instance de l'architecture xUnit pour les frameworks de tests unitaires : Perl, C++, Python, Visual Basic, C#, etc.
- Les **générateurs** de tests JUnit font désormais partie de nombreux **IDE Java** (Eclipse, IntelliJ IDE, NetBeans et Visual Studio Code) et outils de création (Gradle, Maven et Ant).
- Junit est un framework **de tests de régression pour Java**.

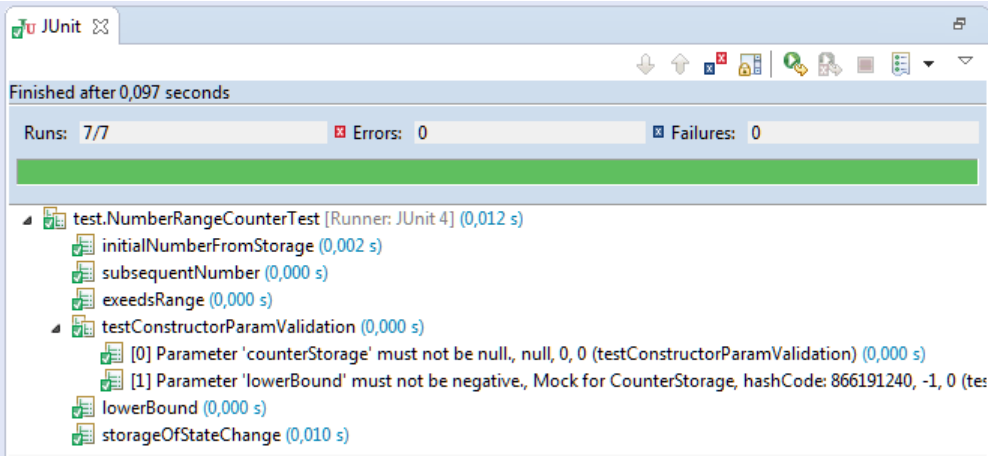
Exemple : ancienne méthode vs. JUnit

- ```
  int max(int a, int b) {
      if (a > b) {
          return a;
      } else {
          return b;
      }
  }
  ```
- ```
  void testMax() {
      int x = max(3, 7);
      if (x != 7) {
          System.out.println("max(3, 7) gives " + x);
      }
      x = max(3, -7);
      if (x != 3) {
          System.out.println("max(3, -7) gives " + x);
      }
  }
  ```
- ```
  public static void main(String[] args) {
      new MyClass().testMax();
  }
  ```

```
@Test
void testMax() {
    assertEquals(7, max(3, 7));
    assertEquals(3, max(3, -7));
}
```
```

Qu'est-ce que JUnit ?

- Les cas de test et les résultats de test sont **des objets Java.**
- Les fonctionnalités JUnit incluent :
  - Des assertions pour tester les résultats attendus
  - Des fonctionnalités de test pour partager des données de test communes
  - Des suites de tests pour organiser et exécuter facilement des tests
  - Des exécuteurs de tests graphiques et textuels



Rédaction de tests pour JUnit

- Chaque méthode de test vérifie une condition (assertion) et signale au testeur si le test a échoué ou réussi
- Le testeur utilise le résultat pour signaler à l'utilisateur (en mode ligne de commande) ou mettre à jour l'affichage (dans un IDE)
- Toutes les méthodes renvoient **void**
- Exemples de méthodes représentatives :
  - *fail (String):* échoue intentionnellement au cas de test
  - *assertTrue (boolean) :* affirme que la condition booléenne passée est vraie
  - *assertTrue (Boolean, String) :* même assertTrue (boolean) mais ajoute une chaîne facultative qui sera affichée en cas d'échec
  - *assertEquals (expected, actual, String) :* affirme que l'attendu et l'actuel sont égaux.

Écrire des tests pour JUnit

- Un dispositif de test (**test fixture**) est l'état du test
  - Objets et variables utilisés par plusieurs tests
  - Initialisations (valeurs préfixées)
  - Réinitialisation des valeurs (valeurs postfixées)
- Différents tests peuvent utiliser les objets sans partager l'état

- Les objets utilisés dans les tests doivent être déclarés comme variables d'instance
  - Ils doivent être initialisés dans une méthode @BeforeEach
  - Peut être désalloué ou réinitialisé dans une méthode @AfterEach

Exemples :

- Si vous testez un code qui met à jour l'enregistrement d'un employé, vous avez besoin d'un enregistrement d'employé pour le tester.

Écriture des test JUnit

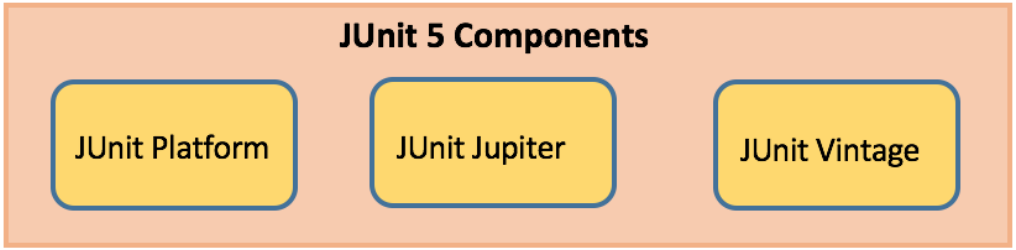
- Séparation de l'initialisation (et de la destruction) du test et des tests
- Utilisé pour garantir qu'il n'y a pas d'effets secondaires entre les tests.
- Application de la règle d'indépendance des tests, l'ordre d'exécution des tests n'est pas garanti.
- Il n'est pas certain que JUnit exécute les tests dans un ordre spécifique.
  - L'exécution des tests sur différentes plateformes et machines virtuelles Java peut donc produire des résultats différents.

Exemple simple :

```java
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }
}
```

```java
//JUnit 5 Test case design for Calculator java class
import org.junit.jupiter.api.*;

public class CalculatorJUnit5Test {

    private Calculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator();
        System.out.println("Setting up for test");
    }

    @AfterEach
    void tearDown() {
        calculator = null;
        System.out.println("after test);
    }

    @Test
    void testAddition() {
        System.out.println("addition test");
        assertEquals(4, calculator.add(2, 2));
    }

    @Test
    void testSubtraction() {
        System.out.println("subtraction test");
        assertEquals(2, calculator.subtract(4, 2));
    }
}
```

Architecture de JUnit 5



- **Plateforme JUnit :**
  - Lancement de frameworks de test sur la JVM
  - Dispose d'une API TestEngine utilisée pour créer un framework de test qui s'exécute sur la plateforme JUnit
- **JUnit Jupiter**
  - Mélange d'un nouveau modèle de programmation pour l'écriture de tests et d'un modèle d'extension
  - Ajout de nouvelles annotations comme @BeforeEach, @AfterEach, etc.
- **JUnit Vintage**
  - Fournit un support pour exécuter les tests des versions 3 et 4 précédentes de JUnit sur cette nouvelle plateforme

JUnit 5: Test simple

Une classe de test (appelée également conteneur)

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import example.util.Calculator;

import org.junit.jupiter.api.Test;

class MyFirstJUnitJupiterTests {

    private final Calculator calculator = new Calculator();

    @Test
    void addition() {
        assertEquals(2, calculator.add(1, 1));
    }

}
```

L'annotation @Test indique qu'une méthode est une méthode de test.

Chaque méthode de test doit être évaluée par rapport à la condition true à l'aide **d'assertions** afin que le test puisse continuer à s'exécuter.

L'assertion assertEquals(attendu, actuelle) échoue lorsque la valeur attendue n'est pas égale à la valeur réelle

Exemple de test TriType (1)

```java
package org.example;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

class TriTypeTest {

    @Test
    void testEquilateralTriangle() {
        assertEquals(1, TriType.classifyTriangle(3, 3, 3), "Failed to identify equilateral triangle");
    }

    @Test
    void testIsoscelesTriangle() {
        assertEquals(2, TriType.classifyTriangle(5, 5, 8), "Failed to identify isosceles triangle");
        assertEquals(2, TriType.classifyTriangle(7, 10, 7), "Failed to identify isosceles triangle");
        assertEquals(2, TriType.classifyTriangle(10, 8, 8), "Failed to identify isosceles triangle");
    }
```

Exemple de test TriType (2)

```
    @DisplayName("Invalid: sum of 2 sides less than third side")
    @Test
    void testInvalidTriangleSumIssue() {
        assertThrows(IllegalArgumentException.class, () -> TriType.classifyTriangle(1, 2, 3),
                "Failed to throw exception for invalid triangle");
    }

    @DisplayName("Invalid: negative side")
    @Test
    void testInvalidTriangleNegativeSide() {
        assertThrows(IllegalArgumentException.class, () -> TriType.classifyTriangle(-1, 4, 5),
                "Failed to throw exception for negative sides");
    }
    @DisplayName("Invalid: side zero")
    @Test
    void testInvalidTriangleSideZero() {
        assertThrows(IllegalArgumentException.class, () -> TriType.classifyTriangle(0, 5, 5),
                "Failed to throw exception for zero-length side");
    }
}
```

JUnit 5 : Annotations de base

- @BeforeEach : Denotes that the annotated method should be executed *before* **each** @Test method in the current class.
- @AfterEach : Denotes that the annotated method should be executed *after* **each** @Test method in the current class.
- @BeforeAll Denotes that the annotated method should be executed *before* **all** @Test methods in the current class;
- @AfterAll : Denotes that the annotated method should be executed *after* **all** @Test methods in the current class.
- @Disabled : Used to disable a test class or test method. (analogous to JUnit 4's @Ignore)
- @DisplayName Declares a custom display name for the test class or test method.
- @Nested denotes that the annotated class is a nested, non-static test class
- @Tag declares tags for filtering tests

Annotations de JUnit 5 : @BeforeAll, @BeforeEach, @Test, @Disabled, @AfterEach, @AfterAll

```java
class StandardTests {

    @BeforeAll
    static void initAll() {
    }

    @BeforeEach
    void init() {
    }

    @Test
    void succeedingTest() {
    }

    @Test
    void failingTest() {
        fail("a failing test");
    }

    @Test
    @Disabled("for demonstration purposes")
    void skippedTest() {
        // not executed
    }

    @Test
    void abortedTest() {
        assumeTrue("abc".contains("Z"));
        fail("test should have been aborted");
    }

    @AfterEach
    void tearDown() {
    }

    @AfterAll
    static void tearDownAll() {
    }

}
```

- assumeTrue est utilisé pour garantir qu'un test ne s'exécute que **si une certaine condition est vraie.**
- Si la condition est fausse, le test **est ignoré (pas échoué).**

Junit 5 : annotation @DisplayName

```
class StandardTests {

    @BeforeAll
    static void initAll() {
    }

    @BeforeEach
    void init() {
    }

    @Test
    void succeedingTest() {
    }

    @Test
    void failingTest() {
        fail("a failing test");
    }

    @Test
    @Disabled("for demonstration purposes")
    void skippedTest() {
        // not executed
    }
```

```
    @Test
    @DisplayName("╯°□°╯")
    void testWithDisplayNameContainingSpecialCharacters() {
    }

    @Test
    @DisplayName("😱")
    void testWithDisplayNameContainingEmoji() {
    }
```

```
    @Test
    void abortedTest() {
        assumeTrue("abc".contains("Z"));
        fail("test should have been aborted");
    }

    @AfterEach
    void tearDown() {
    }

    @AfterAll
    static void tearDownAll() {
    }

}
```

JUnit 5: Assertions

Toutes les assertions JUnit Jupiter sont des méthodes statiques dans la classe org.junit.jupiter.api.Assertions.

```
import static java.time.Duration.ofMillis;
import static java.time.Duration.ofMinutes;
import static org.junit.jupiter.api.Assertions.assertAll;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTimeout;
import static org.junit.jupiter.api.Assertions.assertTimeoutPreemptively;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.util.concurrent.CountDownLatch;

import example.domain.Person;
import example.util.Calculator;

import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;
```

Assertions de JUnit 5 : assertEquals, assertTrue, assertAll

```java
class AssertionsDemo {

    private final Calculator calculator = new Calculator();

    private final Person person = new Person("Jane", "Doe");

    @Test
    void standardAssertions() {
        assertEquals(2, calculator.add(1, 1));
        assertEquals(4, calculator.multiply(2, 2),
                "The optional failure message is now the last parameter");
        assertTrue('a' < 'b', () -> "Assertion messages can be lazily evaluated -- "
                + "to avoid constructing complex messages unnecessarily.");
    }

    @Test
    void groupedAssertions() {
        // In a grouped assertion all assertions are executed, and all
        // failures will be reported together.
        assertAll("person",
            () -> assertEquals("Jane", person.getFirstName()),
            () -> assertEquals("Doe", person.getLastName())
        );
    }
```

assertAll()	Regroupez plusieurs assertions et chaque assertion est exécutée même si une ou plusieurs d'entre elles échouent
assertTrue(expression)	Échoue si l'expression n'est pas vraie

Assertions JUnit 5 : assertNull, assertNotNull

```java
void dependentAssertions() {
    // Within a code block, if an assertion fails the
    // subsequent code in the same block will be skipped.
    assertAll("properties",
        () -> {
            String firstName = person.getFirstName();
            assertNotNull(firstName);

            // Executed only if the previous assertion is valid.
            assertAll("first name",
                () -> assertTrue(firstName.startsWith("J")),
                () -> assertTrue(firstName.endsWith("e"))
            );
        },
        () -> {
            // Grouped assertion, so processed independently
            // of results of first name assertions.
            String lastName = person.getLastName();
            assertNotNull(lastName);

            // Executed only if the previous assertion is valid.
            assertAll("last name",
                () -> assertTrue(lastName.startsWith("D")),
                () -> assertTrue(lastName.endsWith("e"))
            );
        }
    );
}
```

assertNull(actual)	Échoue lorsque la valeur réelle n'est pas nulle
assertNotNull(actual)	Échoue lorsque la valeur réelle est nulle

Exceptions dans JUnit 5

```
 int factorial (int x) {
// EFFECTS: if x > 0 then returns x! else returns an
exception
  if (x < 0) throw new IllegalArgumentException("x must
be >= 0");
  int fact = 1;
  while (x > 1) { fact = fact * x; x = x - 1; } ;
    return fact;
}
```

- In Junit 5:

```
@Test
void testFactNegative() {
assertThrows(IllegalArgumentException.class,
     () -> test.factorial(-12));
}
```

Assertions de JUnit 5 : assertThrows, assertTimeout

```
@Test
void exceptionTesting() {
    Exception exception = assertThrows(ArithmeticException.class, () ->
        calculator.divide(1, 0));
    assertEquals("/ by zero", exception.getMessage());
}

@Test
void timeoutNotExceeded() {
    // The following assertion succeeds.
    assertTimeout(ofMinutes(2), () -> {
        // Perform task that takes less than 2 minutes.
    });
}

@Test
void timeoutNotExceededWithResult() {
    // The following assertion succeeds, and returns the supplied object.
    String actualResult = assertTimeout(ofMinutes(2), () -> {
        return "a result";
    });
    assertEquals("a result", actualResult);
```

assertThrows() La classe à tester devrait générer une exception

- La fonction assertTimeout() permet de vérifier qu'un morceau de code spécifique termine son exécution dans un délai spécifié.
- Si le code prend plus de temps que le temps autorisé, le test échoue.
- Méthodes de test impliquant des **calculs complexes ou des tâches potentiellement longues.**

Assertions JUnit 5 : assertTimeoutPreemptively

```java
@Test
void timeoutNotExceededWithMethod() {
    // The following assertion invokes a method reference and returns an object.
    String actualGreeting = assertTimeout(ofMinutes(2), AssertionsDemo::greeting);
    assertEquals("Hello, World!", actualGreeting);
}

@Test
void timeoutExceeded() {
    // The following assertion fails with an error message similar to:
    // execution exceeded timeout of 10 ms by 91 ms
    assertTimeout(ofMillis(10), () -> {
        // Simulate task that takes more than 10 ms.
        Thread.sleep(100);
    });
}

@Test
void timeoutExceededWithPreemptiveTermination() {
    // The following assertion fails with an error message similar to:
    // execution timed out after 10 ms
    assertTimeoutPreemptively(ofMillis(10), () -> {
        // Simulate task that takes more than 10 ms.
        new CountDownLatch(1).await();
    });
}
```

- Contrairement à assertTimeout(), assertTimeoutPreemptively() interrompt l'exécution du code testé si le délai d'attente est dépassé.

- **Assurez-vous que le code répond à des exigences de performances strictes.**

Hypothèses JUnit 5 : assumeTrue, assumeFalse, assumeThat

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assumptions.assumeTrue;
import static org.junit.jupiter.api.Assumptions.assumingThat;

import example.util.Calculator;

import org.junit.jupiter.api.Test;

class AssumptionsDemo {

    private final Calculator calculator = new Calculator();

    @Test
    void testOnlyOnCiServer() {
        assumeTrue("CI".equals(System.getenv("ENV")));
        // remainder of test
    }

    @Test
    void testOnlyOnDeveloperWorkstation() {
        assumeTrue("DEV".equals(System.getenv("ENV")),
            () -> "Aborting test: not on developer workstation");
        // remainder of test
    }
```

```java
    @Test
    void testInAllEnvironments() {
        assumingThat("CI".equals(System.getenv("ENV")),
            () -> {
                // perform these assertions only on the CI server
                assertEquals(2, calculator.divide(4, 2));
            });

        // perform these assertions in all environments
        assertEquals(42, calculator.multiply(6, 7));
    }

}
```

Si une hypothèse échoue,
une TestAbortedException est levée et le test **est simplement ignoré, sans échec.**

assumingThat permet d'exécuter conditionnellement des assertions ***basées sur une hypothèse****. Si cette condition est vraie, le deuxième argument (une expression lambda) sera exécuté.*

JUnit 5 Tagging and Filtering

- Le marquage (à l'aide de l'annotation @Tag) permet d'attribuer une ou plusieurs balises aux méthodes de test ou aux classes de test.

- Catégorisez les tests pour faciliter l'identification et le filtrage.

- Exécution de tests avec des balises (à l'aide de maven) :

mvn test -Dtest=SomeTest#testFastMethod -Dtags=fast

- Exclure le test avec des balises (en utilisant maven) :

mvn test -Dtags="~slow" # Excludes tests tagged as "slow"

- Combinaison de balises et de filtres à l'aide de la logique AND ou OR (à l'aide de Maven) :

mvn test -Dtags="fast,performance”
mvn test -Dtags="~slow,~integration"

```
@Tag("fast")
@Test
void testFastMethod() {
   // test code here
}

@Tag("slow")
@Test
void testSlowMethod() {
   // test code here
}
```

JUnit 5 : Ordre de test : @TestMethodOrder, @Order

```java
import org.junit.jupiter.api.MethodOrderer.OrderAnnotation;
import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;

@TestMethodOrder(OrderAnnotation.class)
class OrderedTestsDemo {

    @Test
    @Order(1)
    void nullValues() {
        // perform assertions against null values
    }

    @Test
    @Order(2)
    void emptyValues() {
        // perform assertions against empty values
    }

    @Test
    @Order(3)
    void validValues() {
        // perform assertions against valid values
    }

}
```

- @TestMethodOrder contrôle l'ordre d'exécution des méthodes de test dans une classe.
- @Order spécifie l'ordre dans lequel les tests individuels sont exécutés.

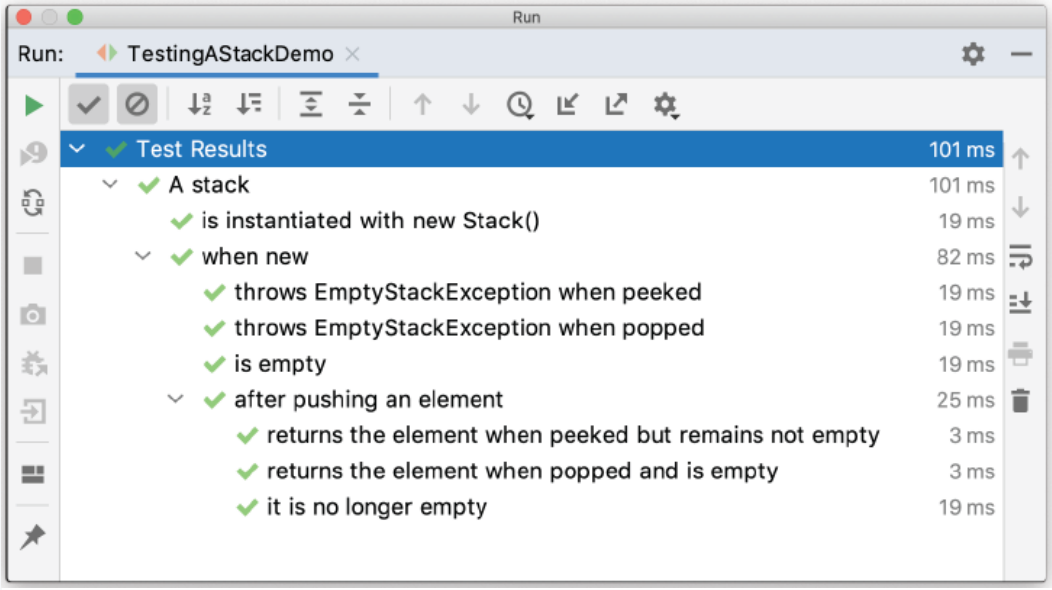
JUnit 5 : Tests imbriqués : @Nested

- Les tests imbriqués permettent de créer des classes imbriquées et d'exécuter toutes leurs méthodes de test.
- **Les classes internes doivent être non statiques.**
- Il suffit d'annoter les classes internes avec **@Nested** et toutes les méthodes de test qu'elles contiennent seront exécutées.

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.util.EmptyStackException;
import java.util.Stack;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;

@DisplayName("A stack")
class TestingAStackDemo {

    Stack<Object> stack;

    @Test
    @DisplayName("is instantiated with new Stack()")
    void isInstantiatedWithNew() {
        new Stack<>();
    }

    @Nested
    @DisplayName("when new")
    class WhenNew {

        @BeforeEach
        void createNewStack() {
            stack = new Stack<>();
        }

        @Test
        @DisplayName("is empty")
        void isEmpty() {
            assertTrue(stack.isEmpty());
        }

        @Test
        @DisplayName("throws EmptyStackException when popped")
        void throwsExceptionWhenPopped() {
            assertThrows(EmptyStackException.class, stack::pop);
        }

        @Test
        @DisplayName("throws EmptyStackException when peeked")
        void throwsExceptionWhenPeeked() {
            assertThrows(EmptyStackException.class, stack::peek);
        }
```

JUnit 5 : Tests imbriqués : @Nested

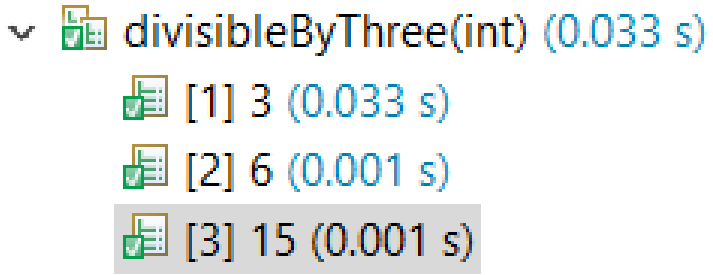
```java
@Nested
@DisplayName("after pushing an element")
class AfterPushing {

    String anElement = "an element";

    @BeforeEach
    void pushAnElement() {
        stack.push(anElement);
    }

    @Test
    @DisplayName("it is no longer empty")
    void isNotEmpty() {
        assertFalse(stack.isEmpty());
    }

    @Test
    @DisplayName("returns the element when popped and is empty")
    void returnElementWhenPopped() {
        assertEquals(anElement, stack.pop());
        assertTrue(stack.isEmpty());
    }

    @Test
    @DisplayName("returns the element when peeked but remains not empty")
    void returnElementWhenPeeked() {
        assertEquals(anElement, stack.peek());
        assertFalse(stack.isEmpty());
```



Tests paramétrés JUnit 5

- Paramètre unique avec @ValueSource (fonctionne lorsque la méthode de test prend un argument)

```
import org.junit.jupiter.params.ParameterizedTest;
Import org.junit.jupiter.params.provider.ValueSource;

@ParameterizedTest
@ValueSource(ints = { 3, 6, 15})
void divisibleByThree(int number) {
    assertEquals(0, number % 3);
}
```



Tests paramétrés JUnit 5

- **@MethodSource** fait référence à une méthode factory qui renvoie les arguments, tels que Stream, Iterable, Iterator ou un tableau d'arguments.

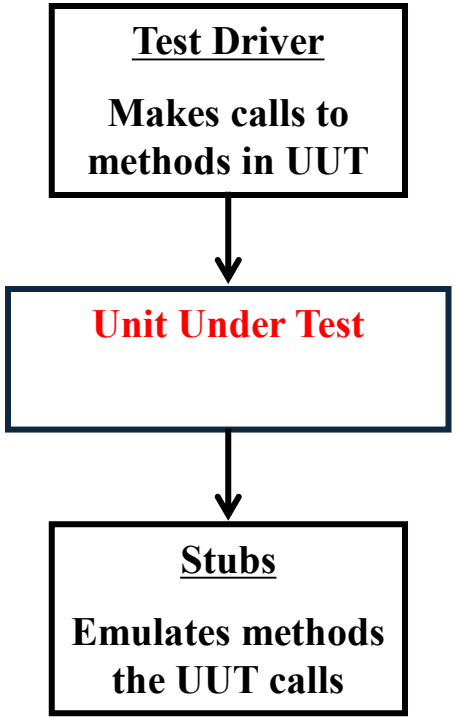
```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.MethodSource;
@ParameterizedTest
@MethodSource("FactorialPairs")
void FactorialTest(int res, int input) {
    assertEquals(res, tester.fact(input));
}
private static Stream<Arguments> FactorialPairs() {
    return Stream.of(
    Arguments.of(1, 1),
    Arguments.of(2, 2),
    Arguments.of(6, 3)
    );
}
```

FactorialTest(int, int) (0.004 s)
- [1] 1, 1 (0.004 s)
- [2] 2, 2 (0.002 s)
- [3] 6, 3 (0.002 s)

Stubs et drivers : contexte d'un test unitaire

- Identifier l'unité testée (UUT)

- Émuler l'environnement de test :
  - un appelant de l'unité, appelé driver (pilote de test).
  - toutes les **unités appelées** par l'unité, appelées stubs.
- Préparez l'UUT :
  - Isolez l'UUT de son environnement.
  - Faites de l'UUT un module exécutable indépendamment en ajoutant un stub et un pilote de test.
- Les drivers et les stubs sont appelés ensemble *scaffolding* (échafaudage)



Drivers (Pilotes de tests)

- De nombreux programmeurs ajoutent des pilotes à chaque classe par habitude
  - La forme la plus simple de pilote est une méthode main() contenant des instructions qui effectuent des tests simples de la classe
  - Instanciez des objets et effectuez des tests simples
- Les pilotes de test peuvent être facilement créés automatiquement
- Les valeurs peuvent être codées en dur ou lues à partir de fichiers
- Par exemple, si la classe est un ADT (Abstract Data Type), le driver de test main() créera certains objets de la classe, ajoutera des valeurs, récupérera des valeurs et utilisera les autres opérations de la classe

Stubs

- La première responsabilité d'un stub est de permettre à la CUT d'être compilée et liée sans erreur
  - La signature doit correspondre
- Que se passe-t-il si la méthode appelée doit renvoyer des valeurs ?
- Ces valeurs ne seront pas les mêmes que celles renvoyées par la méthode complète
- Il peut être important pour les tests qu'elles satisfassent à certaines contraintes limitées

Stubs

- Approches :
  - Renvoyer des valeurs constantes à partir du stub
  - Renvoyer des valeurs aléatoires
  - Renvoyer des valeurs à partir d'une recherche dans une table
  - Renvoyer des valeurs saisies par le testeur pendant l'exécution
  - Traitement des spécifications formelles de la méthode stubbed

Plus coûteux / plus efficace

Stubs vs. Mocks

- Un stub ou un mock remplace un objet qui n'est pas disponible pendant un test.
- Ils isolent tous deux le code testé en remplaçant les dépendances par des implémentations contrôlées.
- Un stub sert de remplacement simpliste d'une unité indisponible
  - En général, il fournit une implémentation simpliste de l'objet indisponible. Lorsqu'une requête est envoyée à un stub, elle peut renvoyer une valeur prédéfinie au lieu de la calculer ou de se connecter à une base de données.
  - Les valeurs renvoyées par un stub sont ensuite vérifiées pour s'assurer de leur exactitude et utilisées dans des tests ultérieurs.
  - Les stubs ne vérifient pas la manière dont ils sont appelés ; ils renvoient simplement les réponses prédéfinies.
  - Cela est également connu sous le nom de vérification d'état.
- Pour la vérification d'état, JUnit propose l'instruction « assert » pour vérifier si une valeur renvoyée par une unité testée est correcte ou non.

Stubs vs. Mocks

- Les mocks sont utilisées pour vérifier les interactions entre le code testé et ses dépendances.
  - Assurez-vous que certaines méthodes **sont appelées** avec **les arguments attendus.**
  - Les simulations sont utiles lorsque vous devez vérifier que le code testé **interagit avec ses dépendances de la manière attendue.**
  - Ceci est également connu sous le nom de vérification du comportement.

Mockito

- Cadre de simulation open source pour Java
  - Utilisé en conjonction avec des cadres de test tels que JUnit.

- Permet aux développeurs de:

1) Créer des instances fictives de classes ou d'interfaces sans avoir besoin de leurs implémentations réelles.
2) Définir comment un mock doit répondre aux appels de méthode avec des arguments spécifiques.
3) Vérifier si des méthodes spécifiques ont été appelées avec les paramètres attendus.

Ajout de dépendances Mockito à Maven pom.xml

```
<dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>

   <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.12.0</version>
      <scope>test</scope>
    </dependency>

  <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-junit-jupiter</artifactId>
      <version>5.12.0</version>
      <scope>test</scope>
   </dependency>
</dependencies>
```

Tests unitaires avec Mockito : un exemple simple

- **TaxRateProvider** est une **dépendance** (peut provenir de la configuration, de la base de données ou d'un service distant).

TaxRateProvider.java

```java
package com.example;

public interface TaxRateProvider {
  /** e.g., 0.15 means 15% tax */
  double taxRate();
}
```

Injection de dépendance (constructor injection): `InvoiceCalculator` reçoit un `TaxRateProvider` via son constructeur. Il ne le crée pas lui-même (`new ...`), donc on peut lui donner un mock en test.

InvoiceCalculator.java (class metier)

```java
package com.example;

public class InvoiceCalculator {
  private final TaxRateProvider taxRateProvider;

  public InvoiceCalculator(TaxRateProvider taxRateProvider) {
    this.taxRateProvider = taxRateProvider;
  }

  public double total(double subtotal) {
    if (subtotal < 0) {
      throw new IllegalArgumentException("subtotal must be >= 0");
    }
    double rate = taxRateProvider.taxRate();
    return subtotal + (subtotal * rate);
  }
}
```

Mockito contrôle le taux d'imposition et nous vérifions l'appel + le total calculé.

Tests unitaires avec Mockito : un exemple simple

MockitoExtension “branche” Mockito à JUnit 5. Il initialise automatiquement les champs annotés @Mock.

Mockito crée un objet qui “implémente” TaxRateProvider sans vraie logique.
Par défaut, un mock retourne des valeurs “vides” (pour double ce serait 0.0), sauf si on définit un comportement.

Stubbing”: définir le comportement du mock. On dit : **quand** on appelle taxRate(), **alors** retourne 0.15.

Vérification des interactions (verify): On vérifie que InvoiceCalculator.total(...) a bien consulté le taux de taxe.

Vérification du résultat (assert)

InvoiceCalculatorTest.java

```java
12  @ExtendWith(MockitoExtension.class)
13  class InvoiceCalculatorTest {
14
15    @Mock
16    TaxRateProvider taxRateProvider;
17
18    @Test
19    void total_appliesTaxRate() {
20      when(taxRateProvider.taxRate()).thenReturn(0.15); // 15%
21
22      InvoiceCalculator calc = new InvoiceCalculator(taxRateProvider);
23
24      double result = calc.total(subtotal: 100.00);
25
26      // verify we asked the dependency for the rate
27      verify(taxRateProvider).taxRate();
28
29      // 100 + 100*0.15 = 115
30      assertEquals(115.00, result, 1e-9);
31    }
32
33    @Test
34    void total_zeroSubtotal_isZeroEvenWithTax() {
35      when(taxRateProvider.taxRate()).thenReturn(0.20);
36
37      InvoiceCalculator calc = new InvoiceCalculator(taxRateProvider);
38
39      assertEquals(0.0, calc.total(subtotal: 0.0), 1e-9);
40      verify(taxRateProvider).taxRate();
41    }
42  }
```

Tests unitaires avec Mockito : un exemple simple

InvoiceCalculatorTest.java

```java
47  @Test
48  void total_negativeSubtotal_throwsIllegalArgumentException() {
49      InvoiceCalculator calc = new InvoiceCalculator(taxRateProvider);
50
51      IllegalArgumentException ex = assertThrows(
52          IllegalArgumentException.class,
53          () -> calc.total(-1.0)
54      );
55
56      // Optionnel : vérifier le message (si tu veux être strict)
57      assertTrue(ex.getMessage().contains("subtotal must be >= 0"));
58
59      // Important : aucune interaction avec la dépendance
60      verifyNoInteractions(taxRateProvider);
61  }
```

Le test doit vérifier deux choses :

1. **Une exception est bien levée**
2. **La dépendance n'est pas appelée** (parce qu'on sort avant taxRateProvider.taxRate())

Différence entre “assert” vs “verify”

- `assertEquals(...)` : vérifie le **résultat** (le “quoi”).
- `verify(...)` : vérifie le **comportement** (le “comment”, ici : “a-t-il demandé le taux ?”).

- Dans beaucoup de tests, `assert` suffit.
- `verify` est utile quand :
- On veut s’assurer qu’une dépendance est appelée (ex: envoyer un email),
- On veut vérifier le nombre d’appels, ou les arguments.

Monkey Testing

- Les entrées sont générées de manière aléatoire **sans structure ni modèle spécifique.**
- Il ne nécessite pas de conception de cas de test détaillé, ce qui le rend rapide à mettre en œuvre.
- **Peut révéler des problèmes inattendus que des tests structurés pourraient manquer.**
- Aide à comprendre comment le système se comporte dans des conditions imprévisibles.
- Le caractère aléatoire peut conduire à manquer certains scénarios de test importants (manque d'assurance de couverture).

Monkey Testing avec JUnit 5

Utiliser des bibliothèques pour la génération de données aléatoires :

- **Apache Commons Lang pour les utilitaires :**

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.12.0</version>
</dependency>
```

- **Java Faker** pour des données aléatoires réalistes :

```
<dependency>
  <groupId>com.github.javafaker</groupId>
  <artifactId>javafaker</artifactId>
  <version>1.0.2</version>
</dependency>
```

Monkey Testing avec JUnit 5 : Générateur de données aléatoires

```
// Use Java's Random class

import java.util.Random;
public class RandomInputGenerator {
   private static final Random RANDOM = new
Random();

   public static int randomInt() {
      return RANDOM.nextInt();
   }
   // Add more methods as needed for different
data types
}
```

```
// Using Apache Commons Lang
import org.apache.commons.lang3.RandomStringUtils;

public class RandomInputGenerator {
   public static String randomAlphanumeric(int length) {
      return
RandomStringUtils.randomAlphanumeric(length);
   }
   // Add more methods as needed
}
```

```
// Using Java Faker
import com.github.javafaker.Faker;
public class RandomInputGenerator {
   private static final Faker faker = new Faker();
   public static String randomName() {
      return faker.name().fullName();
   }
   public static String randomAddress() {
      return faker.address().fullAddress();
   }
   // Add more methods as needed
}
```

Monkey Testing avec JUnit 5 : Exemple

```
public class Calculator {
   public int add(int a, int b) {
      return a + b;
   }
   // Other methods...
}
```

```
import java.util.Random;

public class RandomInputGenerator {
   private static final Random RANDOM =
new Random();

   public static int randomInt() {
      return RANDOM.nextInt();
   }
}
```

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class CalculatorMonkeyTest {
   private Calculator calculator;
   @BeforeEach
   public void setup() {
      calculator = new Calculator();
   }
   @Test
   public void testAddWithRandomInputs() {
      for(int i = 0; i < 1000; i++) {
         int a = RandomInputGenerator.randomInt();
         int b = RandomInputGenerator.randomInt();
         int expected = a + b;
         int result = calculator.add(a, b);
         assertEquals(expected, result, "Addition failed for: " + a + ", "
+ b);
      }
   }
   // Additional test methods for other operations
}
```

References

- www.junit.org
- https://junit.org/junit5/
- https://www.arhohuttunen.com/junit-5-parameterized-tests/